

LP Relaxation for Training Binary Neural Networks

Li-Ping Liu

Introduction

In this work, we study methods for training large-scale Binary Neural Networks (BNNs). BNNs have the potential to significantly reduce the energy consumption of traditional neural networks that run calculations on floating-point numbers.

We focus on Pure Binary Neural Networks (PBNN), which run ALL calculations with binary operations and do *not* use any integers or floating-point numbers. This is different from many BNN studies, where a small fraction of operations still run on floating-point or integer arithmetic.

The focus on PBNN is motivated by hardware design. If a large-scale neural network (e.g. an LLM) can be implemented using pure binary operations, then the hardware design would be greatly simplified. The calculation can then easily be run on an FPGA or an In-Memory Computing (IMC) device.

At the same time, training a PBNN is notoriously hard. A PBNN is fundamentally a function of a large number of binary variables, and training a PBNN is essentially optimizing this function with respect to binary model parameters. It is well known that finding the global optimum is NP-hard and hopeless in general. However, the theory of binary optimization suggests that the optimization problem is not as hard when there is a large number of optimization variables, which gives the problem more freedom. Meanwhile, research on deep learning shows that a learning model still works well even when its parameters do not form the global optimum of the loss function.

This work aims to develop a new optimization method for PBNN by combining stochastic training with a Linear-Programming (LP) relaxation of the binary optimization problem.

Related Work

[TODO: add a discussion of related work.]

Method

Setting up the learning problem

We assume an input instance is $\mathbf{x} \in \{0, 1\}^d$, and the class label is $y \in \{1, \dots, C\}$. We consider a feedforward neural network parameterized by:

- $W^1, W^2, W^3, \dots, W^T$, with each $W^t \in \{0, 1\}^{d_t \times d_{t-1}}$; and
- $b^1, b^2, b^3, \dots, b^T$, with each $b^t \in \{0, 1\}^{d_t}$.

Let $\odot$, $+$, and $\oplus$ respectively represent the AND, OR, and XOR operations. Let $\bigvee(\cdot)$ denote the or-reduce function, which takes OR over a list of variables.

$$\bigvee(z_1, \dots, z_n) = z_1 + z_2 + \dots + z_n.$$

Let $\mathbf{h}^0 = \mathbf{x}$, and each layer is computed by:

$$h_i^t = b_i^t \oplus \bigvee(W_{i,j}^t \odot h_j^{t-1} : j = 1, \dots, d_{t-1})$$

In the last layer, the prediction is $\mathbf{s} = \mathbf{h}^T$. We obtain the loss by comparing $\mathbf{s}$ against y ; here the computation is no longer limited to binary operations. We assume that the loss on this instance is:

$$L = \sum_{j=1}^{d_T} u_j s_j$$

Here u_j is a float constant decided by y . [TODO: need more work to properly define u_j from y].

From the perspective of computation, we could run the data loading with a CPU and then send the data to a special device running PBNN to get the prediction $\mathbf{s}$. Then the CPU can run further calculations to get the final prediction $\hat{y}$.

Minimize the objective with the LP relaxation

Now we want to minimize the loss with respect to binary model parameters (W matrices and b vectors). LP relaxation is a well-known method for optimizing objectives defined on binary variables. To relax the problem into a continuous problem, we also include $\mathbf{h}^t$ for all t as variables.

First, we need to express the relationship $\mathbf{h}^t$, $\mathbf{W}^t$, and $\mathbf{h}^{t-1}$ as a linear constraint. Let g_i^t be the value after OR-reduce but before the XOR operation with b_i^t ,

$$g_i^t = \bigvee_{j=1}^{d_{t-1}} (W_{i,j}^t \odot h_j^{t-1}), \quad h_i^t = b_i^t \oplus g_i^t.$$

Relax all variables to the unit interval, with $\mathbf{h}^0 = \mathbf{x}$ fixed:

$$W_{i,j}^t \in [0, 1], \quad b_i^t \in [0, 1], \quad g_i^t \in [0, 1], \quad h_i^t \in [0, 1]$$

For $t = 1, \dots, T$, $i = 1, \dots, d_t$, $j = 1, \dots, d_{t-1}$, introduce $z_{i,j}^t = W_{i,j}^t \otimes h_j^{t-1}$. We linearize the AND operation with the (two-sided) McCormick envelope:

$$z_{i,j}^t \leq W_{i,j}^t, \quad z_{i,j}^t \leq h_j^{t-1}, \quad z_{i,j}^t \geq W_{i,j}^t + h_j^{t-1} - 1, \quad z_{i,j}^t \geq 0$$

The OR-reduce operation can be relaxed to:

$$g_i^t \geq z_{i,j}^t \quad \forall j, \quad g_i^t \leq \sum_{j=1}^{d_{t-1}} z_{i,j}^t$$

The linearization of $h_i^t = b_i^t \oplus g_i^t$ is:

$$h_i^t \leq g_i^t + b_i^t, \quad h_i^t \leq 2 - g_i^t - b_i^t, \quad h_i^t \geq g_i^t - b_i^t, \quad h_i^t \geq b_i^t - g_i^t$$

The resulting LP is:

$$\begin{aligned} \min_{W, b, g, h, z} \quad & L = \sum_{j=1}^{d_T} u_j h_j^T \\ \text{s.t.} \quad & z_{i,j}^t \leq W_{i,j}^t, \quad z_{i,j}^t \leq h_j^{t-1}, \quad z_{i,j}^t \geq W_{i,j}^t + h_j^{t-1} - 1, \quad z_{i,j}^t \geq 0 \quad \forall t, i, j \\ & g_i^t \geq z_{i,j}^t \quad \forall t, i, j, \quad g_i^t \leq \sum_{j=1}^{d_{t-1}} z_{i,j}^t \quad \forall t, i \\ & h_i^t \leq g_i^t + b_i^t, \quad h_i^t \leq 2 - g_i^t - b_i^t, \quad h_i^t \geq g_i^t - b_i^t, \quad h_i^t \geq b_i^t - g_i^t \quad \forall t, i \\ & \mathbf{h}^0 = \mathbf{x}, \quad W_{i,j}^t \in [0, 1], \quad b_i^t \in [0, 1], \quad g_i^t \in [0, 1] \end{aligned}$$

Stochastic training

Let $\alpha = (W^1, \dots, W^T, b^1, \dots, b^T)$ denote the shared parameters, with $|\alpha| = T(d^2 + d)$ under the $d_t = d$ simplification. For a training set of N samples $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}$, let

$$\beta^{(k)} = \left\{ z_{i,j}^{t,(k)}, g_i^{t,(k)}, h_i^{t,(k)} \right\}_{t,i,j}$$

denote the per-sample auxiliary variables introduced by the linearization above, with $h^{0,(k)} = \mathbf{x}^{(k)}$ fixed; each $\beta^{(k)}$ has $T(d^2 + 2d)$ entries. The full LP is

$$\min_{\alpha, \beta^{(1)}, \dots, \beta^{(N)}} \sum_{k=1}^N \sum_{j=1}^{d_T} u_j h_j^{T,(k)} \quad \text{s.t.} \quad \text{constraints}(\alpha, \beta^{(k)}; \mathbf{x}^{(k)}) \quad \forall k = 1, \dots, N,$$

where “constraints” collects the AND, OR-reduce, and XOR linearizations from above, applied independently to each sample. This has $|\alpha| + N \cdot |\beta^{(1)}|$ variables in total.

Exact decomposition. The constraint blocks for different samples share only α and never interact directly, so the minimization over $\beta^{(1)}, \dots, \beta^{(N)}$ separates exactly:

$$\begin{aligned} & \min_{\alpha, \beta^{(1:N)}} \sum_{k=1}^N \sum_j u_j h_j^{T,(k)} \quad \text{s.t.} \quad \text{constraints}(\alpha, \beta^{(k)}; \mathbf{x}^{(k)}) \quad \forall k = 1, \dots, N \\ &= \min_{\alpha} \sum_{k=1}^N \underbrace{\min_{\beta^{(k)}} \left\{ \sum_j u_j h_j^{T,(k)} : \text{constraints}(\alpha, \beta^{(k)}; \mathbf{x}^{(k)}) \right\}}_{=: V(\alpha; \mathbf{x}^{(k)})} \\ &= \min_{\alpha \in [0,1]^{|\alpha|}} \sum_{k=1}^N V(\alpha; \mathbf{x}^{(k)}). \end{aligned}$$

The only constraint on α is that it lies in the box $[0, 1]^{|\alpha|}$. Therefore, we only need a subgradient of $\sum_{k=1}^N V(\alpha; \mathbf{x}^{(k)})$ to get the update direction for the parameters in α . If an update step would move a parameter outside $[0, 1]$, the parameter is instead projected back onto the boundary.

The subgradient can be estimated using $V(\alpha; \mathbf{x}^{(k)})$ evaluated on a subset (mini-batch) of training examples. For each sample, we first compute the optimal $\beta^{(k)}$ (i.e. solve for $V(\alpha; \mathbf{x}^{(k)})$), and then take the subgradient of the result with respect to α .